

VLAN hopping

Follow-along: play the attacker, then the defender

Overview

Definition: VLAN (virtual LAN)

A partition of one physical switched network into separate broadcast domains. Each port is assigned to a VLAN, and a switch forwards a frame only to ports in the frame's own VLAN, so two hosts on the same switch behave as if they were on two switches with no cable between them. In this lab VLAN 10 holds the attacker and a peer, VLAN 20 holds the victim, and no router joins them.

Definition: VLAN hopping

Getting a frame delivered into a VLAN the sender's own port does not belong to. In the form this lab builds, the sender never gains a route or an address in the target VLAN; it gets one frame at a time across a boundary the switch is supposed to enforce. Here the attacker sits in VLAN 10 and lands ICMP echo requests on a host in VLAN 20.

A **VLAN** is the boundary a great many networks rely on to keep a guest wireless segment away from finance, a lab bench away from the servers it borrows a switch from, or a building's cameras away from its workstations. The boundary is enforced by the switch, not by a router or a firewall, so a mistake in one switch port's settings is worth as much to an attacker as a routing mistake, and it is visible to nothing that inspects routed traffic.

This lab breaks that boundary by **VLAN hopping**: a frame the attacker crafts with two VLAN tags stacked in it, sent from a port that carries only VLAN 10, arrives at a host in VLAN 20. The break needs no exploit, no password, and no reply from the victim. Part 2 closes it twice over, and each fix is a change to a single switch port setting.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one lab per row. Select **VLAN hopping** to open its actions:

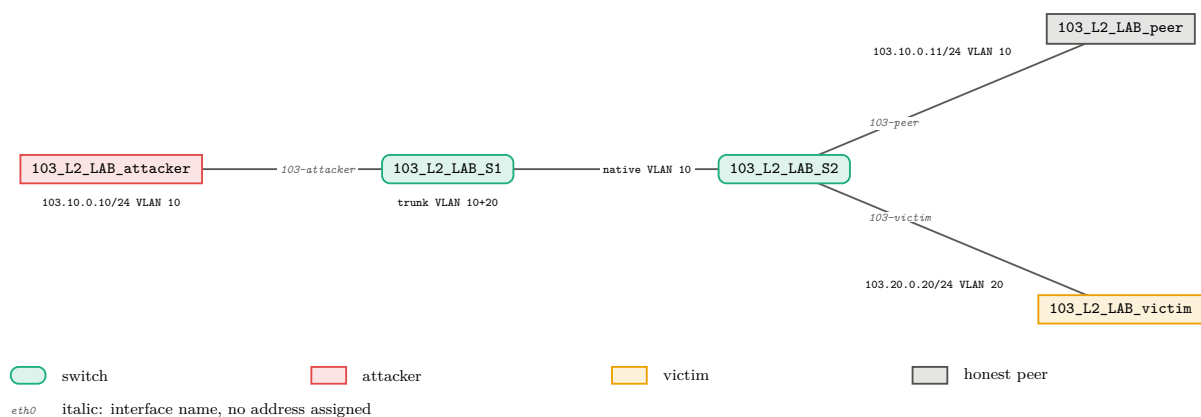
- **Spawn** creates this lab's two switches, its three hosts, and the link between the switches. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.

- **Status** prints each switch's per-port VLAN settings, whether the attacker can still reach the host in its own VLAN, and whether a hopped frame is reaching the victim in the other one. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** asks which node (attacker, peer, victim, S1, S2) and prints the `docker exec -it ...` line to paste into your own terminal; each step below names the node it runs on.
- **Reset to baseline** stops the attack and undoes your switch changes, returning the lab to its clean Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup

Three hosts on two switches, with one link between the switches carrying both VLANs:



Each node is labelled with the container name you pass to `docker exec`, each host link with that host's address and VLAN, and each switch-side end with the switch port name. The same addressing, to copy from:

Role	IP	VLAN	Switch	Container
attacker	103.10.0.10	10	S1	103_L2_LAB_attacker
peer	103.10.0.11	10	S2	103_L2_LAB_peer
victim	103.20.0.20	20	S2	103_L2_LAB_victim

Definition: 802.1Q tag

Four bytes inserted into an Ethernet frame after the source MAC address, carrying the VLAN id the frame belongs to. A frame with no tag carries no VLAN of its own and takes the VLAN of the port it arrives on. You will measure the four bytes in step 6.

Definition: Access port and trunk port

An access port connects one host and carries one VLAN. Frames leave it with no 802.1Q tag, because the host behind it is not expected to know VLANs exist. A trunk port connects two switches and carries several VLANs at once, tagging each frame so the switch at the far end can tell which VLAN it belongs to; step 2 adds the one exception to that tagging rule. Here **103-attacker** on S1 is a host port in VLAN 10, and the S1-to-S2 link is a trunk carrying VLAN 10 and VLAN 20.

The switches are **103_L2_LAB_S1** and **103_L2_LAB_S2**, joined by one **trunk**. Every host NIC is named after the switch it plugs into: **103-S1** on the attacker, **103-S2** on the peer and the victim. Each switch port is named after what is on the other end, so S1's trunk port is **103-S2** and S2's is **103-S1**. That means **103-S2** is a host NIC when you are inside the victim and a trunk port when you are inside S1; every command block below names the container it is typed in, and the interface follows from that. There is no router anywhere in this lab, so nothing forwards between 103.10.0.0/24 and 103.20.0.0/24 at the IP layer.

The attacker carries a stock Layer 2 kit; there is no bespoke exploit to run:

- **ip** (iproute2): reads the interfaces, and in step 4 adds an 802.1Q subinterface for the single-tag attempt.
- **scapy**: builds a frame one header at a time, so you set each VLAN tag yourself.
- **tcpdump**: on the attacker, on either switch, and on the victim. Running it at three points along one path is how you watch a tag disappear.

The switches run Open vSwitch, and **ovs-vsctl** reads and writes their port settings. Open a shell wherever a step says so with:

```
# your own terminal: <container> is any container name from the table above
docker exec -it <container> bash
```

Assessment. This lab is graded on sixteen questions, marked inline below as **Question N**, each placed at the step whose output it asks about. Answer all sixteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step; none needs anything beyond the lab in front of you.

Part 1: the attack

You start with an address in VLAN 10 and no way to reach VLAN 20. Steps 1 to 4 establish what the switches will and will not carry for you. Steps 5 to 8 put one frame into the victim's VLAN and follow it, header by header, to see which switch did what.

1. **Try to reach both other hosts.** On the attacker, ping the peer and then the victim, and read the second failure carefully:

```
# 103_L2_LAB_attacker: the attacker, VLAN 10
docker exec -it 103_L2_LAB_attacker bash
ping -c2 103.10.0.11          # the peer, VLAN 10, on the far switch
ping -c2 -W1 103.20.0.20     # the victim, VLAN 20
ip route                     # what the attacker knows how to reach
```

Question 1. The ping to the victim ends in `ping: connect: Network unreachable` rather than in lost packets. Using the output of `ip route`, say what that error tells you about the attacker's own configuration, and state why it is not yet evidence that the switches keep VLAN 10 and VLAN 20 apart.

2. **Read the switch's VLAN plan.** The hop turns on how three ports are configured, so read them before touching anything. Each `ovs-vsctl get` prints the three values on their own lines, in the order asked for:

```
# 103_L2_LAB_S1: the switch the attacker plugs into
docker exec -it 103_L2_LAB_S1 bash
ovs-vsctl get port 103-attacker vlan_mode tag trunks # the attacker's port
ovs-vsctl get port 103-S2          vlan_mode tag trunks # the trunk toward S2
```

Definition: Native VLAN

The one VLAN a trunk carries without a tag. A frame in the native VLAN leaves the trunk with no 802.1Q tag on it, and a frame arriving on the trunk with no tag is treated as belonging to the native VLAN. Every other VLAN on that trunk is carried tagged. Open vSwitch keeps the native VLAN in the `tag` column of a port whose `vlan_mode` is `native-untagged` or `native-tagged`, and keeps the port's one VLAN there when the mode is `access`. The mode spelled `trunk` has no native VLAN at all and its `tag` must be empty, so a switch-to-switch link that sends one VLAN untagged is `native-untagged`, not `trunk`.

Both ports report `vlan_mode` of `native-untagged`. On such a port, `trunks` lists the VLANs the port will carry, and a frame tagged with anything outside that list is dropped as it arrives. An empty `trunks` column is the exception: it carries every VLAN rather than none, and neither of these two ports is set that way. `tag` holds the **native VLAN**. Read both ports' three values and compare the two `tag` values before you answer.

Question 2. From the values you just read, predict what happens to each of these two frames as it leaves S1 on the trunk toward S2: one belonging to VLAN 10, and one belonging to VLAN 20. Say which crosses the trunk carrying an 802.1Q tag and which crosses it without one, and name the single value you read that decides it.

3. **Check the prediction on honest traffic.** You do not have to take the last answer on trust. Capture on S1's trunk port while ordinary VLAN 10 traffic crosses it. In one terminal:

```
# 103_L2_LAB_S1: capture on the trunk port toward S2
docker exec -it 103_L2_LAB_S1 bash
tcpdump -e -n -i 103-S2 icmp # leave running
```

In a second terminal, make the attacker talk to the peer, which is legitimate VLAN 10 traffic between two hosts in the same VLAN:

```
# 103_L2_LAB_attacker: one command, run from your own terminal
docker exec 103_L2_LAB_attacker ping -c2 103.10.0.11
```

Read the trunk capture: `-e` prints the Ethernet header, so a tag would appear as a `vlan 10` field between the MAC addresses and `ethertype IPv4`. Note the frame length `tcpdump`

reports; you will compare it against the same traffic under a different setting in Part 2.

4. **Tag one frame for VLAN 20, and watch where it stops.** The obvious attack is to claim VLAN 20 directly. Give the attacker an 802.1Q subinterface for VLAN 20 and an address in the victim's subnet, then try to reach the victim through it. Start both captures first, in two terminals:

```
# 103_L2_LAB_attacker: capture on the attacker's own NIC
docker exec -it 103_L2_LAB_attacker bash
tcpdump -e -n -i 103-S1          # leave running
```

```
# 103_L2_LAB_S1: capture on the trunk toward S2
docker exec -it 103_L2_LAB_S1 bash
tcpdump -e -n -i 103-S2          # leave running
```

Then, in a third terminal, build the subinterface and use it:

```
# 103_L2_LAB_attacker: the single-tag attempt
docker exec -it 103_L2_LAB_attacker bash
ip link add link 103-S1 name 103-S1.20 type vlan id 20
ip addr add 103.20.0.66/24 dev 103-S1.20
ip link set 103-S1.20 up
ping -c3 -W1 103.20.0.20
ip link del 103-S1.20           # remove it before step 5
```

The ping reports three packets sent and none received. Compare your two captures: one of them holds frames tagged `vlan 20`, including an ARP request asking who has `103.20.0.20`, and the other does not.

Question 3. State which of the two captures showed the VLAN 20 frames and which stayed empty, and name the port and the specific setting on it that separates the two. Then say what would have to change about that port for a single VLAN 20 tag to reach the victim, and why an operator would not configure it that way for a host port.

5. **Stack two tags in one frame.** Your port refuses a VLAN 20 tag but accepts a VLAN 10 tag, and the trunk beyond it carries both VLANs. Build a frame that is VLAN 10 on the outside and VLAN 20 on the inside.

Definition: Double tagging

Putting two 802.1Q tags in one frame, one after the other. On a trunk or native-untagged port, a switch decides a frame's VLAN from the outermost tag only, so as far as the first switch is concerned the second tag is payload, no different from the bytes of the IP header behind it.

Start a capture on the victim first, so nothing arrives before you are watching. Every capture from here on runs without a filter, because a filter like `icmp` looks for the IP header at a fixed offset from the start of the frame and can miss a frame that carries tags in front of it. The lab is otherwise idle, so an occasional IPv6 router solicitation is the only other traffic you will see:

```
# 103_L2_LAB_victim: the victim, VLAN 20
docker exec -it 103_L2_LAB_victim bash
tcpdump -e -n -i 103-S2          # leave running
```

Then send the **double-tagged** frame from the attacker, naming every field yourself:

```
# 103_L2_LAB_attacker: one command, run from your own terminal
docker exec 103_L2_LAB_attacker python3 -c "from scapy.all import *; \
    sendp(Ether(src=get_if_hwaddr('103-S1'), dst='ff:ff:ff:ff:ff:ff')/ \
        Dot1Q(vlan=10)/Dot1Q(vlan=20)/ \
        IP(src='103.10.0.10', dst='103.20.0.20')/ICMP(), \
        iface='103-S1', count=5, inter=0.2)"
```

Each field, and why it is what it is:

- `Dot1Q(vlan=10)` is the outer tag, set to the VLAN your own port carries, which is what gets the frame accepted at 103-attacker at all.
- `Dot1Q(vlan=20)` is the inner tag, naming the VLAN you want the frame delivered in.
- `Ether(dst="ff:ff:ff:ff:ff:ff")` is the broadcast address, so the frame needs no knowledge of the victim's MAC address.
- `IP(src=...)` is the attacker's real VLAN 10 address, left honest so that what arrives in VLAN 20 is recognisably from outside it.
- `count=5` sends five copies, enough to read a pattern rather than a single event.

The victim's capture fills with ICMP echo requests from 103.10.0.10. A VLAN 10 address is sending traffic that a host in VLAN 20 receives, with no router in the lab and no route between the two subnets.

6. **Follow the frame and find the missing tag.** Run the same send with a capture at three points at once, in three terminals, so you see the frame leave, cross the trunk, and arrive:

```
# 103_L2_LAB_attacker: point 1, the attacker's own NIC
docker exec -it 103_L2_LAB_attacker bash
tcpdump -e -n -i 103-S1
```

```
# 103_L2_LAB_S1: point 2, the trunk port toward S2
docker exec -it 103_L2_LAB_S1 bash
tcpdump -e -n -i 103-S2
```

```
# 103_L2_LAB_victim: point 3, the victim's NIC
docker exec -it 103_L2_LAB_victim bash
tcpdump -e -n -i 103-S2
```

Send the frame again from a fourth terminal, with the command from step 5. Then read the three captures side by side: how many `vlan` fields each line shows, which VLAN ids, and the `length` tcpdump prints for the frame.

Question 4. Give the frame length reported at each of the three capture points, and the difference between consecutive points. State what that difference is the size of, and confirm it against the number of `vlan` fields you counted at each point.

Question 5. Between point 1 and point 2 the frame lost its outer tag, and no switch is configured to edit tags. Explain what removed it, tying your answer to the specific value you read on 103-S2 in step 2. Then say why the inner VLAN 20 tag was still there to become the frame's only tag.

Question 6. S1 accepted your frame and put it on the trunk, but S1 never delivered it into VLAN 20, even though S1 had the VLAN 20 tag in its hands the whole time. Explain why, using the order in which a switch decides a frame's VLAN and removes a tag. Then state what S2 sees arriving and why it has no way to tell that frame apart from one a genuine VLAN 20 host sent.

7. **Change the outer tag and predict the result.** The outer tag was VLAN 10 for a reason. Before running anything, write down your prediction for what the three captures will show if the outer tag is 99 instead. Then send it and check:

```
# 103_L2_LAB_attacker: same frame, outer tag changed to 99
docker exec 103_L2_LAB_attacker python3 -c "from scapy.all import *; \
    sendp(Ether(src=get_if_hwaddr('103-S1'), dst='ff:ff:ff:ff:ff:ff')/ \
        Dot1Q(vlan=99)/Dot1Q(vlan=20)/ \
        IP(src='103.10.0.10', dst='103.20.0.20')/ICMP(), \
        iface='103-S1', count=5, inter=0.2)"
```

Question 7. Give your prediction, then the three captures you actually got, and account for any difference between them. Name the port that decided the outcome and the value on it that did so, and state the general condition the outer tag has to satisfy for this attack to work at all.

Question 8. Your frame is addressed to the broadcast MAC address rather than to the victim's. Use what you saw in step 4 to explain why the attacker cannot simply look the victim's MAC address up first, the way it would for a host in its own VLAN. Then state what the broadcast address costs the attacker, given that VLAN 20 may hold hosts other than the victim.

8. **Check what the victim sends back.** Capture everything on the victim, not just ICMP, so you see both directions while the attack runs:

```
# 103_L2_LAB_victim: capture every frame, in both directions
docker exec -it 103_L2_LAB_victim bash
tcpdump -n -i 103-S2
```

Send the double-tagged frame from step 5 again and read what the victim's NIC carries.

Question 9. State whether any echo reply appears in the victim's capture. Give two independent reasons the attacker gets nothing back: one from the victim's own routing table (check it with `ip route` on the victim), and one about the path a reply would have to take through the two switches, whatever the victim's routing table said.

Question 10. Take your answer to the previous question as the attacker’s position. Explain what specifically stops the attacker completing a TCP three-way handshake with a service in VLAN 20 from there, and give the general test that follows for deciding whether a given attack can be run from that position at all. Name one kind of action that passes your test.

9. **Confirm the boundary is still there.** So there is no doubt the hop carried the frame rather than a stray route:

```
# 103_L2_LAB_attacker: nothing here has changed since step 1
docker exec 103_L2_LAB_attacker ip route
docker exec 103_L2_LAB_attacker ping -c1 -W1 103.20.0.20
```

Question 11. The attacker’s routing table has no entry for 103.20.0.0/24 and an ordinary ping to the victim still fails, yet five frames reached it. Explain why a firewall placed between the two subnets would not have stopped this, and what that says about the layer at which the fix has to go.

Part 2: the defence

The hop needed two things to be true at once: the attacker’s port had to accept a frame that already carried a tag, and the trunk had to remove exactly one tag on the way out. Break either one and the attack stops. Both fixes below are switch port settings, applied on the switches rather than on the hosts, because the hosts did nothing wrong and a host cannot be trusted to police its own frames anyway.

Your target is the same in both cases: the victim’s capture stays empty while the attacker fires, and the attacker’s ordinary VLAN 10 traffic to the peer still works. That second half is what stops “unplug the attacker” from counting as a defence.

Option A: refuse tagged frames on host ports

Definition: Strict access port

An Open vSwitch port with `vlan_mode=access`. It carries the one VLAN in its `tag` column, and it drops any arriving frame that carries an 802.1Q tag, whatever VLAN the tag names, including its own. The port the attacker used was `native-untagged` instead, which accepts both untagged frames and frames tagged with a VLAN the port trunks.

The attacker’s frame got in because `103-attacker` accepted a tagged frame at all. A host port has no reason to: the machine behind it runs one operating system on one subnet and has no business naming a VLAN. Make it a **strict access port**:

```
# 103_L2_LAB_S1: lock the attacker's port to one VLAN, untagged only
docker exec -it 103_L2_LAB_S1 bash
ovs-vsctl set port 103-attacker vlan_mode=access tag=10
ovs-vsctl clear port 103-attacker trunks
ovs-vsctl get port 103-attacker vlan_mode tag trunks
```

Now re-run the attack from step 5 with captures at the attacker’s NIC, S1’s trunk port, and the victim, and re-run the honest `ping -c2 103.10.0.11` from the attacker.

Question 12. Give the three captures under this fix, and say at which of the three points the frame stops. The peer's port and the victim's port on S2 were already `vlan_mode=access` in the baseline you started from. Explain why the attack worked anyway, and what that says about which ports in a network this fix has to be applied to.

Question 13. The attacker's untagged VLAN 10 traffic to the peer still passes while every tagged frame it sends is dropped. Explain why that split costs the attacker its hop and costs an ordinary VLAN 10 host nothing, and name one kind of legitimate device on a host port that this fix would break.

Option B: move the native VLAN off every VLAN in use

Undo Option A first, with **Reset to baseline** in the TUI or `scripts/reset.sh`, so you are testing this fix on its own.

The second precondition was the tag removal on the trunk, and that happened only because the trunk's native VLAN, 10, was also the attacker's own VLAN. Park the native VLAN on an id no host uses. It has to be set on both ends of the trunk, because each end decides for itself what it sends untagged:

```
# 103_L2_LAB_S1: S1's end of the trunk
docker exec -it 103_L2_LAB_S1 bash
ovs-vsctl set port 103-S2 tag=999
exit
```

```
# 103_L2_LAB_S2: S2's end of the trunk, set to match
docker exec -it 103_L2_LAB_S2 bash
ovs-vsctl set port 103-S1 tag=999
ovs-vsctl get port 103-S1 vlan_mode tag trunks
```

Re-run the attack from step 5. This time capture at four points, and label each terminal before you send anything, because three of the four watch an interface called 103-S2 in three different containers: 103_L2_LAB_attacker on 103-S1, 103_L2_LAB_S1 on 103-S2, which is S1's trunk port, 103_L2_LAB_victim on 103-S2, and 103_L2_LAB_peer on 103-S2. The first three are the captures from step 6; the fourth is new:

```
# 103_L2_LAB_peer: the honest VLAN 10 host on the far switch
docker exec -it 103_L2_LAB_peer bash
tcpdump -e -n -i 103-S2
```

Then run the honest `ping -c2 103.10.0.11` again with S1's trunk capture running, and compare the frame length on the trunk against the length you noted in step 3.

Question 14. Walk the double-tagged frame from the attacker to its final destination under this setting. Say how many tags it carries on the trunk, which host's capture it turns up in, why that host is the one that receives it, and why nothing in VLAN 20 sees it. Then give the honest ping's frame length on the trunk now against step 3's, and account for the difference.

Question 15. The standard hardening advice is "never leave the native VLAN at the default, VLAN 1". The native VLAN here was 10, not 1, and moving it to 999 fixed the hop anyway.

State the general rule that VLAN 1 advice is one instance of: what must the native VLAN never be equal to, and why does that equality, rather than the number 1, create the hole?

What each fix guards

Question 16. Option A refuses the tagged frame where it enters the network; Option B removes the tag removal it depends on. Consider a network that runs strict access ports on every host port but leaves a trunk's native VLAN equal to a VLAN its hosts use. Say whether this attack still works, and answer separately for an attacker on a host port and for an attacker who has reached a trunk port, for example by unplugging a switch and taking its place. Then say which of the two fixes you would apply first if you could only apply one, and why.

Checks: what “done” means

- **Attack works:** with the baseline settings in place, the victim's `tcpdump -e -n -i 103-S2` shows echo requests from 103.10.0.10 arriving with no VLAN tag on them, while an ordinary `ping` from the attacker to 103.20.0.20 still fails at the attacker's own routing table.
- **Defence holds:** with either fix in place, the same send leaves the victim's capture empty, and `ping -c2 103.10.0.11` from the attacker to the peer still answers.

`scripts/status.sh` prints each switch's per-port VLAN settings, the attacker-to-peer reach, and whether a hopped frame is still landing on the victim, so you can read the before and after without shelling in. `scripts/reset.sh` returns the ports to the Part 1 baseline.

Safety. Everything here lives on two isolated Open vSwitch bridges with no route off the host. The double-tagged frames only ever reach the lab's own containers, and the attacker has no address on any network outside them.

When you are done, tear the lab down. Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left running on your host. **Reset to baseline** only rewinds the attack and the defence; only **Teardown** deletes the lab.